МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт № 8 «Компьютерные науки»

ЛАБОРАТОРНАЯ РАБОТА № 3

по курсу «Программирование графических процессоров»

«Классификация и кластеризация изображений на GPU»

Выполнил: Д. А. Кулешов
Группа: М8О-408Б-23
Преподаватель: А. Ю. Морозов

Москва 2026

1 Условие

Лабораторная работа №3

Классификация и кластеризация изображений на GPU.

Цель работы. Научиться использовать GPU для классификации и кластеризации изображений. Использование *константной памяти* и *одномерной сетки потоков*.

Формат изображений соответствует формату описанному в лабораторной работе 2. Во всех вариантах, в результирующем изображении, на месте альфа-канала должен быть записан номер класса(кластера) к которому был отнесен соответствующий пиксель. Если пиксель можно отнести к нескольким классам, то выбирается класс с наименьшим номером.

В вариантах 1-4, формат входных данных одинаковый. На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. На следующей строке, число nc -- количество классов. Далее идут nc строчек описывающих каждый класс. В начале j-ой строки задается число np_j -- количество пикселей в выборке, за ним следуют np_j пар чисел -- координаты пикселей выборки. $nc \leq 32$, $np_j \leq 2^{19}$, $w*h \leq 4 * 10^8$.

Оценка вектора средних и ковариационной матрицы:

$$avg_j = \frac{1}{np_j} \sum_{i=1}^{np_j} ps_i^j$$

$$cov_j = \frac{1}{np_j - 1} \sum_{i=1}^{np_j} (ps_i^j - avg_j) * (ps_i^j - avg_j)^T$$

где $ps_i^j = (r_i^j \; g_i^j \; b_i^j)^T$ -- i-ый пиксель из j-ой выборки.

Рисунок 1 — Общая постановка задачи и формат входных данных

Вариант 3. Метод минимального расстояния.

Для некоторого пикселя p, номер класса jc определяется следующим образом:

$$jc = arg\,max_j\left[-(p - avg_j)^T * (p - avg_j)\right]$$

Пример:

Входной файл	hex: in.data	hex: out.data
in.data out.data 2 4 1 2 1 0 2 2 2 1 4 0 0 0 1 1 1 2 0	03000000 03000000 A2DF4C00 F7C9FE00 9ED84500 B4E85300 99D14D00 92DD5600 A9E04C00 F7D1FA00 D4D0E900	03000000 03000000 A2DF4C01 F7C9FE00 9ED84501 B4E85301 99D14D01 92DD5601 A9E04C01 F7D1FA00 D4D0E900

Входной файл	hex: out.data
in.data out.data 5 4 5 0 0 2 6 1 1 1 6 2 0 7 1 1 0 1 2 6 0 4 0 4 3 0 3 1 0 1 0 0 4 0 3 6 2 5 2 7 2 9 6 4 5 1 7 0 2 1 2 3 4 1 1 5 3 3 2 6	08000000 08000000 D2E27502 CFF65201 D3ED5701 D6E76902 C8F35B01 8E168200 CFF45001 AE977604 D3DC7102 7D1E7B00 AB9A8004 D9E58602 AB967E04 AE9D8004 87058200 D0F95B01 74148000 D0F55901 86136C00 85077400 D6E27702 D3609F03 D1609F03 CC5EA103 CC739D03 7C127F00 AA988804 AFA07D04 D0E37702 7D117A00 D6EB5901 D6E37C02 C9F85701 D655A103 D7EA7402 93127D00 D35BA403 D4DD7902 B0A18404 D6DE7502 D765A903 AD928404 D0D87C02 D7E97F02 CD509E03 CAF85201 CFF75601 CEF45E01 D0E86902 D1D17F02 AD928104 AFA18304 D4DB5C01 88077D00 C6F75701 7D127D00 A99A8E04 C8609E03 D15DA503 AB957E04 AE9A8004 79218100 D065A103 A99E9A04

Рисунок 2 — Условие варианта 3 — метод минимального расстояния

2 Программное и аппаратное обеспечение

2.1 Аппаратное обеспечение

Характеристики использованного аппаратного обеспечения приведены в таблице 1.

Таблица 1 — Характеристики аппаратного обеспечения

Характеристика	Значение
Процессор	AMD Ryzen 5 5600, 6 ядер, 12 потоков
Кэш процессора	L1d — 192 KiB; L1i — 192 KiB; L2 — 3 MiB; L3 — 32 MiB
Оперативная память	32 GiB; в WSL доступно 15 GiB; swap 4 GiB
Накопитель	INTEL SSDPEKNU512GZ, 512 GB
Графический процессор	NVIDIA GeForce RTX 5070

Характеристика	Значение
Compute capability	12.0
Графическая память	12 227 MiB
Разделяемая память на блок	49 152 байт (48 KiB)
Константная память	65 536 байт (64 KiB)
Регистров на блок	65 536
Максимальное число нитей в блоке	1024
Максимальный размер сетки	2 147 483 647 × 65 535 × 65 535 блоков
Число мультипроцессоров	48

Характеристики GPU были получены с помощью `nvidia-smi` и функции `cudaGetDeviceProperties`.

2.2 Программное обеспечение

Использованное программное обеспечение приведено в таблице 2.

Таблица 2 — Использованное программное обеспечение

Компонент	Значение
Операционная система	Ubuntu 26.04.1 LTS в WSL
Архитектура	x86-64
CUDA Toolkit	13.4
Компилятор CUDA	nvcc 13.4.92
Драйвер NVIDIA	617.14
Среда разработки	Visual Studio Code

3 Метод решения

Для каждого класса j по заданной выборке пикселей вычисляется средний вектор цвета

$$\text{avg}_j = \frac{1}{n_{p_j}} \sum_{i=1}^{n_{p_j}} p_i^j, \tag{1}$$

где $p_i^j = \left(r_i^j, g_i^j, b_i^j\right)^T$ — пиксель из выборки класса j, n_{p_j} — размер этой выборки.

Для произвольного пикселя p вычисляется квадрат евклидова расстояния до среднего вектора каждого класса:

$$d_{j(p)} = \left(p - \text{avg}_j\right)^T \left(p - \text{avg}_j\right). \tag{2}$$

Пиксель относится к классу с минимальным расстоянием:

$$j_c = \arg\,\min_j d_{j(p)}. \tag{3}$$

Это равносильно максимизации отрицательного расстояния, указанной в условии. Если минимальное расстояние одинаково для нескольких классов, сохраняется класс с меньшим номером.

Средние векторы вычисляются на CPU и копируются в константную память GPU. Все пиксели хранятся в линейном массиве. Нити одномерной CUDA-сетки обрабатывают их независимо и после каждого пикселя смещаются на полный размер сетки. Номер класса записывается в альфа-канал, компоненты RGB не изменяются. Вычислительная сложность алгоритма составляет $O(whn_c)$.

4 Описание программы

Работа состоит из следующих файлов:

- `src/cuda/lab3.cu` — основная CUDA-программа;
- `src/cuda/benchmark.cu` — CPU-реализация, проверка корректности и измерение производительности разных конфигураций CUDA;
- `src/convert/png2data.py`, `src/convert/data2png.py` и `src/convert/data2classes.py` — преобразование изображений и визуализация номеров классов;
- `Makefile` — сборка программ и отчёта.

Пиксели представлены типом `uchar4`. Средние векторы цветов представлены типом `double3` и хранятся в массиве `class_averages` в константной памяти. Исходное изображение копируется в линейную память GPU и классифицируется на месте.

4.1 CUDA-ядро

Ядро `kernel` вычисляет линейный индекс пикселя по индексам блока и нити. Для пикселя последовательно вычисляются расстояния до средних цветов классов. Используется строгое сравнение `distance < min_distance`, поэтому при равных расстояниях выбирается класс с меньшим номером. Цикл с шагом, равным общему числу нитей, позволяет обрабатывать изображения любого допустимого размера одной конфигурацией запуска.

4.2 CPU-версия

Функция `cpu_classify` из файла `benchmark.cu` реализует тот же алгоритм последовательным циклом. После каждого измерения результаты CPU и GPU сравниваются попиксельно, включая номер класса в альфа-канале.

5 Результаты

5.1 Проведение замеров

Время CUDA-ядра измерялось с помощью CUDA Events, время CPU-версии — с помощью `chrono`. Перед замерами выполнялся прогрев. Копирование данных между CPU и GPU не учитывалось. Для изображений 1024 × 1024, 4096 × 4096 и 8192 × 8192 выполнялось соответственно 50, 10 и 3 повтора. В каждом тесте использовалось пять классов. В таблицах указано среднее время одного запуска. Программы собраны с оптимизацией `-O2`.

5.2 Время работы CUDA-ядра

Результаты измерения времени CUDA-ядра приведены в таблице 3. Единица измерения — миллисекунда.

Таблица 3 — Среднее время работы CUDA-ядра

Конфигурация	1024 × 1024, мс	4096 × 4096, мс	8192 × 8192, мс
`<<<1, 32>>>`	22,765152	440,068939	1768,311523
`<<<32, 32>>>`	0,670309	15,887018	66,206596
`<<<64, 128>>>`	0,242785	4,736803	18,805941
`<<<128, 256>>>`	0,182354	3,520813	14,422027
`<<<256, 256>>>`	0,182604	3,51824	14,433376

Конфигурация	1024 × 1024, мс	4096 × 4096, мс	8192 × 8192, мс
`<<<1024, 1024>>>`	0,172282	3,330221	13,226186

Конфигурация `<<<1, 32>>>` использует только один warp и поэтому плохо подходит для больших изображений. После насыщения GPU дальнейшее увеличение числа нитей почти не сокращает время работы.

5.3 Сравнение с CPU

Ускорение вычисляется по формуле

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}, \tag{4}$$

где T_{CPU} — среднее время последовательной версии, а T_{GPU} — среднее время CUDA-ядра. Для каждого размера в таблице 4 выбрана самая быстрая из измеренных конфигураций.

Таблица 4 — Сравнение времени CPU и лучшей конфигурации GPU

Вход	CPU, мс	GPU, мс	Конфигурация	Ускорение
1024×1024	5,730472	0,172282	`<<<1024, 1024>>>`	33,26
4096×4096	92,167741	3,330221	`<<<1024, 1024>>>`	27,68
8192×8192	417,513642	13,226186	`<<<1024, 1024>>>`	31,57

Результаты CPU и GPU совпали попиксельно. Конфигурация `<<<1, 32>>>` медленнее CPU, но при достаточном числе блоков GPU показывает значительное ускорение. Время копирования памяти в это сравнение не входит.

5.4 Результат обработки изображения

Для проверки использовано изображение размером 800 × 800 пикселей и пять выборок характерных цветов. В выходном файле исходные компоненты RGB сохранены, а номер класса записан в альфа-канал. Поэтому справа на рисунке 3 приведена цветовая визуализация значений альфа-канала: каждому классу соответствует отдельный цвет.



а) исходное изображение, 800 × 800



б) визуализация результата классификации

Рисунок 3 — Результат классификации изображения методом минимального расстояния

6 Выводы

В работе реализована классификация пикселей методом минимального расстояния на GPU. Средние векторы классов размещены в константной памяти, а обработка пикселей распределена между нитями одномерной CUDA-сетки.

Для изображений размером 1024 × 1024, 4096 × 4096 и 8192 × 8192 получено ускорение соответственно в 33,26, 27,68 и 31,57 раза относительно последовательной CPU-версии. Конфигурация с одним блоком не обеспечивает достаточного параллелизма и уступает CPU. Попиксельное совпадение результатов подтвердило корректность CUDA-реализации.

Исходный код работы: https://github.com/dmitriikuleshov/pgp-lab3